Distributed Log Replication System Report

1. Implementation Description

Our project implements a simplified distributed consensus system inspired by the Raft Consensus Algorithm. The system ensures that multiple servers maintain a consistent replicated log despite failures or network delays.

Core Components

Message Class

The `Message` class encapsulates all communication between servers. Each message includes:

- A **label** indicating message type (e.g., `APPEND`, `VOTE`, `HEARTBEAT`)
- A **sequence number** for log ordering
- Source and destination information
- Optional payload (`msg`) or voting flag (`vote_for`) or term

This abstraction allows flexible communication between distributed nodes.

Server Class

Each server is represented with:

- Hostname and port
- Leader status (`is_leader`)

All servers are stored in a shared `table_of_Node`.

System Design

The system is event-driven and multi-threaded, with several concurrent threads:

1. Heartbeat Mechanism

- The followers periodically sends `HEARTBEAT` messages.
- Leader respond with `HEARTBEAT_ACK`.
- If no acknowledgment is received within a timeout, a leader election is triggered.

2. Leader Election

- A server starts an election by sending `VOTE` requests.
- Servers respond with `VOTE_FOR` messages.
- If a majority is reached, the server becomes leader and broadcasts `NEW_LEADER`.

3. Log Replication

- Clients input messages via `keyboard_thread`.
- If the node is leader:
  - It sends `APPEND` messages to followers.
- Followers:
  - Store the log entry
  - Respond with `APPEND_ACK`

4. Commit Mechanism

- Once a majority acknowledges (`APPEND_ACK`), the leader:
  - Sends a `COMMIT` message
- If insufficient acknowledgments:
  - A `REJECT` message is issued

5. Networking

- Communication uses **UDP sockets**
- Messages are serialized using `pickle`
- A proxy server is used for routing messages

---

2. Testing and Experimentation

We tested the system under multiple scenarios:

Test Cases

1. Normal Operation

- Leader successfully replicated logs across all nodes.
- Logs remained consistent across servers.

2. Leader Failure

- When the leader stopped sending heartbeats:
  - Followers triggered election
  - A new leader was elected successfully based on term
- NOTE: Leader is restarted using a separate command line to rejoin the network

3. Network Delay Simulation

- Artificial delays caused:
    - Occasional election triggers
    - Temporary inconsistencies resolved after re-election

4. Partial Acknowledgment

- When fewer than majority acknowledgments were received:
    - Logs were rejected appropriately

Observed Outcomes

- Majority-based commit worked correctly
- Leader election was functional but sensitive to timeout tuning
- System demonstrated eventual consistency

---

3. Evaluation of the Project

Successes

- Successfully implemented:
    - Leader election
    - Heartbeat monitoring
    - Log replication
    - Majority-based commit
- Modular message-based design improved clarity
- Multi-threading enabled realistic distributed behavior

Challenges and Obstacles

1. Concurrency Issues

- Shared variables (e.g., `number_of_votes`, `heartbeat_ack`) were prone to race conditions
- Lack of synchronization mechanisms caused occasional inconsistencies

2. Timeout Tuning

- Choosing appropriate heartbeat and election timeouts was difficult
- Incorrect values caused:
    - Frequent unnecessary elections
    - Parallel elections held between Servers
    - Delayed failure detection

3. UDP Limitations

- UDP does not guarantee delivery
- Required additional logic for reliability (ACKs)

4. Simplifications

Compared to full Raft:

- No persistent storage
- No log consistency checks (e.g., mismatch resolution)

Key Insights

- Distributed systems are highly sensitive to timing
- Majority consensus is critical for consistency
- Failure handling is more complex than normal operation
- Even simplified consensus algorithms require careful design

---

4. References

- Raft Consensus Algorithm — Diego Ongaro and John Ousterhout
- Python documentation for:
    - `socket`
    - `threading`
    - `pickle`

---

5. Group Contributions

Lucan Akpati

- Implemented:
    - Message structure and labeling system

    - Log replication logic (`APPEND`, `COMMIT`, `REJECT`)
    - Keyboard input handling **Samuel Olukuewu**

- Implemented:
    - Leader election and voting system
    - Heartbeat mechanism
    - Failure Handling when leader restarts after crash (NOTE: changed to join system)

Shared Contributions

- System design and architecture
- Testing and experimentation
- Debugging distributed behaviors
- Writing and reviewing documentation

Conclusion

The project successfully demonstrates a working prototype of a distributed log replication system based on Raft principles. While simplified, it highlights the complexity of distributed consensus and provides valuable hands-on experience with fault tolerance, networking, and concurrency.